

Plataforma: GamesBusiness

Greg: Usuario (com autenticação JWT), Biblioteca e Avaliacao.

Fabão: Jogo, Desenvolvedora, Categoria e Conquista.

Requisitos Funcionais

Autenticação e autorização

1. **RF01 – Cadastro de usuário:** qualquer pessoa pode criar uma conta informando nome de usuário, e-mail e senha. Nome de usuário e e-mail são únicos, e a conta nasce com o perfil USER.
2. **RF02 – Login:** o usuário se autentica com e-mail e senha. Se as credenciais forem válidas, o sistema retorna um token JWT.
3. **RF03 – Acesso protegido:** todos os endpoints, exceto cadastro e login, exigem um token válido. Token ausente, inválido ou expirado retorna 401.
4. **RF04 – Controle por perfil:** apenas ADMIN gerencia o catálogo (jogos, categorias, desenvolvedoras e conquistas). Um USER que tentar essas operações recebe 403.

Usuário

5. **RF05 – Gerenciar perfil:** o usuário pode visualizar e editar os próprios dados. O ADMIN pode listar e remover usuários.

Catálogo

6. **RF06 – CRUD de Desenvolvedora (ADMIN):** permite cadastrar, listar, editar e excluir desenvolvedoras. Não é possível excluir uma desenvolvedora que tenha jogos vinculados.
7. **RF07 – CRUD de Categoria (ADMIN):** o nome da categoria é único.
8. **RF08 – CRUD de Jogo (ADMIN):** todo jogo pertence a uma desenvolvedora e tem pelo menos uma categoria.
9. **RF09 – Consultar catálogo:** qualquer usuário autenticado pode listar jogos com paginação e filtrar por título, categoria ou desenvolvedora.
10. **RF10 – CRUD de Conquista (ADMIN):** cada conquista é cadastrada vinculada a um jogo.

Biblioteca

11. **RF11 – Adquirir jogo:** o usuário adiciona um jogo à sua biblioteca, e o sistema registra a data e o preço pago no momento da compra. O mesmo jogo não pode ser adquirido duas vezes.
12. **RF12 – Gerenciar biblioteca:** o usuário lista seus jogos, atualiza as horas jogadas e pode remover um jogo da biblioteca.

Avaliação

13. **RF13 – Avaliar jogo:** o usuário só pode avaliar um jogo que está na sua biblioteca. A nota vai de 1 a 5, com comentário opcional, e cada usuário faz no máximo uma avaliação por jogo.
14. **RF14 – Gerenciar avaliações:** o usuário edita e exclui apenas as próprias avaliações. Qualquer usuário pode ver as avaliações de um jogo e a nota média.

Conquistas do usuário

15. **RF15 – Desbloquear conquista:** o usuário só desbloqueia conquistas de jogos que possui, e cada conquista é desbloqueada uma única vez. O sistema registra a data.
16. **RF16 – Listar conquistas:** o usuário vê suas conquistas desbloqueadas, podendo filtrar por jogo.

Requisitos Não-Funcionais

1. **RNF01 – Plataforma:** o backend é desenvolvido em Java 17 ou superior com Spring Boot 3, Spring Data JPA, Spring Security e Lombok. O Spring Boot 3 exige no mínimo o Java 17.
2. **RNF02 – Banco de dados:** PostgreSQL, em uma versão entre 12 e 17.
3. **RNF03 – Versionamento do banco:** toda alteração de estrutura é feita por migrations do Flyway. Nada é criado pelo `ddl-auto` do Hibernate.
4. **RNF04 – Arquitetura em camadas:** o código é organizado em Controller, Service, Repository, Entity, DTO e ExceptionHandler, e cada camada só conversa com a vizinha.
5. **RNF05 – Segurança das senhas:** as senhas são armazenadas com hash BCrypt e nunca aparecem em nenhuma resposta da API.
6. **RNF06 – Autenticação stateless:** a API não guarda sessão. O JWT tem tempo de expiração definido, por exemplo 2 horas.
7. **RNF07 – Isolamento das entidades:** toda entrada e saída de dados usa DTOs, e as entidades JPA nunca são expostas diretamente.
8. **RNF08 – Validação:** os dados de entrada são validados com Bean Validation (`@NotBlank`, `@Email`, `@Size`, `@Min`, `@Max`). Dados inválidos retornam 400 com a mensagem de cada campo.
9. **RNF09 – Tratamento centralizado de erros:** um `@RestControllerAdvice` captura as exceções e devolve sempre o mesmo formato JSON, com data e hora, status, mensagem e caminho da requisição.
10. **RNF10 – Padrão REST:** a API usa corretamente os verbos HTTP e os códigos de status: 200, 201, 204, 400, 401, 403, 404 e 409 para conflitos como compra duplicada.
11. **RNF11 – Documentação:** os endpoints são documentados com Swagger/OpenAPI (springdoc), e o README traz instruções de execução, estrutura de pastas, exemplos de requisições e credenciais de teste.
12. **RNF12 – Versionamento do código:** o projeto fica em um repositório no GitHub, com commits dos dois integrantes.
13. **RNF13 – Logs (bônus):** operações importantes, como login, compra, exclusões e falhas de autenticação, são registradas com SLF4J.